

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ  
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,  
МЕХАНИКИ И ОПТИКИ»**

**Факультет безопасности информационных технологий**

**Дисциплина:  
«Информационная безопасность баз данных»**

**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3  
Функции и триггеры в БД**

**Выполнил:**

Студент гр. N3248

Жук Анастасия Валерьевна



**Проверил:**

Салихов Максим Русланович

Санкт-Петербург  
2025 г.

**Цель работы:** получение навыков написания процедур, функций и триггеров в БД.

**Задачи:**

1. Изучить теорию о процедурах, функциях и триггерах в языке SQL: способ написания, принцип работы.
2. Выполнить задания, описанные в методическом пособии к лабораторной работе.

## Ход работы

Хранимая процедура — это объект базы данных, представляющий собой набор SQL- инструкций, который компилируется один раз и хранится на сервере.

Хранимая функция отличается от хранимой процедуры, тем что хранимая функция всегда возвращает что-то, а процедура ничего не возвращает.

Триггер (trigger) — это хранимая функция, которую пользователь не вызывает непосредственно, а исполнение которой обусловлено действием по модификации данных: добавлением INSERT, удалением DELETE строки в заданной таблице, или изменением UPDATE данных в определенном столбце заданной таблицы реляционной базы данных. В PostgreSQL триггеры создаются на основе существующих функции, т.е. сначала командой CREATE FUNCTION определяется триггерная функция, затем на ее основе командой CREATE TRIGGER определяется собственно триггер.

Перед тем, как начать работать с процедурами, функциями и триггерами, создадим простенькую БД из 2 таблиц, с которой в дальнейшем и будут производиться манипуляции.

```
CREATE TABLE Authors (  
    author_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Books (  
    book_id SERIAL PRIMARY KEY,  
    title VARCHAR(100) NOT NULL,  
    author_id INTEGER,  
    FOREIGN KEY (author_id) REFERENCES Authors(author_id)  
);
```

*SERIAL* – это способ автоматически генерировать уникальные значения для столбца, который обычно используется как первичный ключ (PRIMARY KEY). Он:

1. Создает последовательность (sequence) — объект базы данных, который генерирует уникальные числа.
2. Назначает эту последовательность столбцу.
3. Устанавливает значение столбца по умолчанию как следующее значение из последовательности.

```
postgres=# CREATE TABLE Authors (  
postgres(#      author_id SERIAL PRIMARY KEY,  
postgres(#      name VARCHAR(100) NOT NULL  
postgres(# );  
CREATE TABLE  
postgres=# CREATE TABLE Books (  
postgres(#      book_id SERIAL PRIMARY KEY,  
postgres(#      title VARCHAR(100) NOT NULL,  
postgres(#      author_id INTEGER,  
postgres(#      FOREIGN KEY (author_id) REFERENCES Authors(author_id)  
postgres(# );  
CREATE TABLE
```

INSERT INTO Authors (name) VALUES

('Лев Толстой'),  
('Фёдор Достоевский'),  
('Антон Чехов'),  
('Михаил Булгаков'),  
('Александр Пушкин');

INSERT INTO Books (title, author\_id) VALUES

('Война и мир', 1),  
('Белые ночи', 2),  
('Пиковая дама', 5),  
('Кавказский пленник', 1),  
('Преступление и наказание', 2),  
('Бедные люди', 2),  
('Вишневый сад', 3),  
('Толстый и тонкий', 3),  
('Униженные и оскорбленные', 2),  
('Роковые яйца', 4),  
('Хамелеон', 3),  
('Мастер и Маргарита', 4),  
('Евгений Онегин', 5),  
('Анна Каренина', 1),  
('Братья Карамазовы', 2),  
('Дама с собачкой', 3),  
('Собачье сердце', 4),  
('Капитанская дочка', 5),  
('Каштанка', 3),  
('Руслан и Людмила', 5);

```
postgres=# SELECT * FROM Authors;
Текущая кодовая страница: 1251
author_id |      name
-----+-----
          1 | Лев Толстой
          2 | Фёдор Достоевский
          3 | Антон Чехов
          4 | Михаил Булгаков
          5 | Александр Пушкин
(5 строк)

postgres=# SELECT * FROM Books;
Текущая кодовая страница: 1251
book_id |      title      | author_id
-----+-----+-----
          1 | Война и мир      |          1
          2 | Белые ночи       |          2
          3 | Пиковая дама     |          5
          4 | Кавказский пленник |          1
          5 | Преступление и наказание |          2
          6 | Бедные люди      |          2
          7 | Вишневый сад     |          3
          8 | Толстый и тонкий |          3
          9 | Униженные и оскорбленные |          2
         10 | Роковые яйца     |          4
         11 | Хамелеон         |          3
         12 | Мастер и Маргарита |          4
         13 | Евгений Онегин   |          5
         14 | Анна Каренина    |          1
         15 | Братья Карамазовы |          2
         16 | Дама с собачкой  |          3
         17 | Собачье сердце   |          4
         18 | Капитанская дочка |          5
         19 | Каштанка         |          3
         20 | Руслан и Людмила |          5
(20 строк)
```

1. Написать процедуру, которая выполняет агрегацию значений в таблице и обновляет значение в другой таблице. Таким образом, чтобы при запуске пользователем информация в таблице обновлялась и содержала агрегированные значения из другой таблицы.

Хотим сделать сводку о том, сколько у каждого автора из Authors книг из Books. Для этого можно создать отдельную таблицу BookCount.

```
CREATE TABLE BookCount (
    author_id INT PRIMARY KEY,
    book_count INT DEFAULT 0,
    FOREIGN KEY (author_id) REFERENCES Authors(author_id)
);
```

А теперь напишем процедуру, которая заполнит таблицу BookCount:

```
CREATE OR REPLACE PROCEDURE update_book_count()
LANGUAGE plpgsql
AS $$
BEGIN
    -- Обновление существующих записей
    UPDATE BookCount bc
    SET book_count = agg.book_count
    FROM (
        SELECT author_id, COUNT(*) AS book_count
        FROM Books
        GROUP BY author_id
    ) agg
    WHERE bc.author_id = agg.author_id;

    -- Добавление новых авторов
    INSERT INTO BookCount (author_id, book_count)
    SELECT agg.author_id, agg.book_count
    FROM (
        SELECT author_id, COUNT(*) AS book_count
        FROM Books
        GROUP BY author_id
    ) agg
    WHERE NOT EXISTS (
        SELECT 1 FROM BookCount bc WHERE bc.author_id = agg.author_id
    );

END;
$;
```

```

postgres=# CREATE OR REPLACE PROCEDURE update_book_count()
postgres=# LANGUAGE plpgsql
postgres=# AS $$
postgres## BEGIN
postgres##
postgres##     UPDATE BookCount bc
postgres##     SET book_count = agg.book_count
postgres##     FROM (
postgres##         SELECT author_id, COUNT(*) AS book_count
postgres##         FROM Books
postgres##         GROUP BY author_id
postgres##     ) agg
postgres##     WHERE bc.author_id = agg.author_id;
postgres##
postgres##     INSERT INTO BookCount (author_id, book_count)
postgres##     SELECT agg.author_id, agg.book_count
postgres##     FROM (
postgres##         SELECT author_id, COUNT(*) AS book_count
postgres##         FROM Books
postgres##         GROUP BY author_id
postgres##     ) agg
postgres##     WHERE NOT EXISTS (
postgres##         SELECT 1 FROM BookCount bc WHERE bc.author_id = agg.author_id
postgres##     );
postgres##
postgres## END;
postgres## $$;
CREATE PROCEDURE

```

Чтобы удалить процедуру, необходимо написать следующую команду:

```
DROP PROCEDURE procedure_name;
```

Чтобы применить процедуру, необходимо написать следующую команду:

```
CALL update_book_count();
```

Так выглядит обновленная таблица BookCount:

```

postgres=# CALL update_book_count();
CALL
postgres=# SELECT * FROM BookCount;
Текущая кодовая страница: 1251
 author_id | book_count
-----+-----
          3 |          5
          5 |          4
          4 |          3
          2 |          5
          1 |          3
(5 строк)

```

2. Написать триггер, который будет выполнять действие из 1 пункта автоматически при вставке записи в исходную таблицу. Таким образом, чтобы агрегированная информация всегда была актуальна.

```

CREATE OR REPLACE FUNCTION trigger_update_book_count()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Вызываем процедуру для обновления статистики
    CALL update_book_count();
    RETURN NEW;
END;
$$;

```

```

-- Создание триггера
CREATE TRIGGER trg_after_insert_book
AFTER INSERT ON Books
FOR EACH ROW
EXECUTE FUNCTION trigger_update_book_count();

```

```

postgres=# CREATE OR REPLACE FUNCTION trigger_update_book_count()
postgres=# RETURNS TRIGGER
postgres=# LANGUAGE plpgsql
postgres=# AS $$
postgres$# BEGIN
postgres$#     CALL update_book_count();
postgres$#     RETURN NEW;
postgres$# END;
postgres$# $$;
CREATE FUNCTION
postgres=# CREATE TRIGGER trg_after_insert_book
postgres=# AFTER INSERT ON Books
postgres=# FOR EACH ROW
postgres=# EXECUTE FUNCTION trigger_update_book_count();
CREATE TRIGGER

```

Ключевое слово *FOR EACH ROW* указывает, что триггер будет выполняться для каждой строки, которая затрагивается операцией (в данном случае INSERT). Это называется строчный триггер. Теперь, когда добавится книгу в таблицу Books, таблица BookCount автоматически обновится.

```

postgres=# INSERT INTO Books (title, author_id) VALUES ('Пари', 3);
INSERT 0 1
postgres=# SELECT * FROM BookCount;
Текущая кодовая страница: 1251

```

| author_id | book_count |
|-----------|------------|
| 3         | 6          |
| 5         | 4          |
| 4         | 3          |
| 2         | 5          |
| 1         | 3          |

(5 строк)



3. Написать триггер, который на основании даты из вставляемой записи, вставлял ее в соответствующую таблицу.

Для этого задания создадим новую БД, в которой будем распределять аудиозаписи по годам: 2025, 2024 и архивные.

```
CREATE TABLE Records (  
    record_id SERIAL PRIMARY KEY,  
    duration TIME NOT NULL,  
    record_date DATE NOT NULL  
);
```

```
CREATE TABLE Records_2025 (  
    LIKE records INCLUDING ALL  
);
```

```
CREATE TABLE Records_2024 (  
    LIKE records INCLUDING ALL  
);
```

```
CREATE TABLE Records_archive (  
    LIKE records INCLUDING ALL  
);
```

Напишем триггер, который поможет распределить аудиозаписи по таблицам:

```
CREATE OR REPLACE FUNCTION records_by_year()  
RETURNS TRIGGER AS $$  
BEGIN  
    -- Проверяем, что дата записи не из будущего  
    IF NEW.record_date > CURRENT_DATE THEN  
        RAISE EXCEPTION 'Дата записи % не может быть в будущем',  
NEW.record_date;  
    ELSIF EXTRACT(YEAR FROM NEW.record_date) = 2025 THEN  
        INSERT INTO Records_2025 VALUES (NEW.*);  
    ELSIF EXTRACT(YEAR FROM NEW.record_date) = 2024 THEN  
        INSERT INTO Records_2024 VALUES (NEW.*);  
    ELSE  
        INSERT INTO Records_archive VALUES (NEW.*);  
    END IF;  
  
    RETURN NULL;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_records_by_year
```

```
BEFORE INSERT ON Records
FOR EACH ROW EXECUTE FUNCTION records_by_year();
```

Добавим записи и посмотрим, что с таблицами:

```
INSERT INTO Records (duration, record_date) VALUES
('01:30:00', '2024-05-15'),
('00:45:00', '2023-11-30'),
('02:00:00', '2026-01-01'),
('02:00:00', '2016-01-01');
```

```
postgres=# INSERT INTO Records (duration, record_date) VA
LUES ('01:30:00', '2024-05-15'), ('00:45:00', '2023-11-30
'), ('02:00:00', '2026-01-01'), ('02:00:00', '2016-01-01'
);
```

ОШИБКА: Дата записи 2026-01-01 не может быть в будущем  
КОНТЕКСТ: функция PL/pgSQL records\_by\_year(), строка 5,  
оператор RAISE

```
postgres=# SELECT * FROM Records_archive;
```

Текущая кодовая страница: 1251

| record_id | duration | record_date |
|-----------|----------|-------------|
| 5         | 00:45:00 | 2023-11-30  |
| 6         | 02:00:00 | 2016-01-01  |

(2 строки)

```
postgres=# SELECT * FROM Records_2024;
```

Текущая кодовая страница: 1251

| record_id | duration | record_date |
|-----------|----------|-------------|
| 4         | 01:30:00 | 2024-05-15  |

(1 строка)

```
postgres=# SELECT * FROM Records_2025;
```

Текущая кодовая страница: 1251

| record_id | duration | record_date |
|-----------|----------|-------------|
|-----------|----------|-------------|

(0 строк)

Сама же таблица Records осталась без записей (так как в функции в конце присутствует RETURN NULL):

```
postgres=# SELECT * FROM Records;
```

Текущая кодовая страница: 1251

| record_id | duration | record_date |
|-----------|----------|-------------|
|-----------|----------|-------------|

(0 строк)

Функция *EXTRACT* — это мощная функция для извлечения отдельных частей из даты, времени или временных меток. Она позволяет получить конкретные компоненты временных данных, такие как год, месяц, день, час и другие.

Также с помощью *RAISE NOTICE* можно выводить дополнительные сообщения.

Для удаления триггера необходимо сделать следующее:

`DROP TRIGGER IF EXISTS trigger_name ON table_name;`

- *DROP TRIGGER* - основная команда для удаления триггера;
- *IF EXISTS* (опционально) - предотвращает ошибку, если триггер не существует;
- `trigger_name` - имя триггера, который нужно удалить;
- `ON table_name` - указывает таблицу, к которой привязан триггер

`DROP FUNCTION IF EXISTS function_name();`

4. Написать триггер, который при вставке в таблицу, производил подмену вставляемого значения в соответствии с уже существующим словарем.

Создадим две таблицы: одна будет словарем, а вторая будет принимать введенные значения. Пусть будет словарь технических устройств.

```
CREATE TABLE Product_dictionary (  
    product_code VARCHAR(10) PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Orders (  
    order_id SERIAL PRIMARY KEY,  
    product_code VARCHAR(10),  
    product_name VARCHAR(100),  
    quantity INTEGER NOT NULL  
);
```

```
INSERT INTO Product_dictionary (product_code, product_name) VALUES  
( 'NB-001', 'Ноутбук Lenovo IdeaPad'),  
( 'NB-002', 'Ноутбук ASUS VivoBook'),  
( 'PH-001', 'Смартфон Samsung Galaxy'),  
( 'PH-002', 'Смартфон iPhone 14'),  
( 'TV-001', 'Телевизор LG OLED');
```

А теперь напишем такой триггер, который будет при добавлении в таблицу `Orders` устройства с кодом менять его на расшифровку из словаря.

```
CREATE OR REPLACE FUNCTION rename_with_dictionary()  
RETURNS TRIGGER AS $$  
BEGIN  
    SELECT product_name INTO NEW.product_name  
    FROM product_dictionary
```

```

WHERE product_code = NEW.product_code;

IF NEW.product_name IS NULL THEN
    RAISE WARNING 'Код товара "%" не найден в словаре', NEW.product_code;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_rename_product_name
BEFORE INSERT ON Orders
FOR EACH ROW EXECUTE FUNCTION rename_with_dictionary();

```

Тесты:

```

INSERT INTO Orders (product_code, quantity) VALUES
('NB-001', 1),
('PH-002', 2),
('TV-001', 1),
('UNKNOWN', 3);

```

```

postgres=# INSERT INTO Orders (product_code, quantity) VALUES
postgres=# ('NB-001', 1),
postgres=# ('PH-002', 2),
postgres=# ('TV-001', 1),
postgres=# ('UNKNOWN', 3);
ПРЕДУПРЕЖДЕНИЕ: Код товара "UNKNOWN" не найден в словаре
INSERT 0 4
postgres=# SELECT * FROM Orders;
Текущая кодовая страница: 1251

```

| order_id | product_code | product_name           | quantity |
|----------|--------------|------------------------|----------|
| 1        | NB-001       | Ноутбук Lenovo IdeaPad | 1        |
| 2        | PH-002       | Смартфон iPhone 14     | 2        |
| 3        | TV-001       | Телевизор LG OLED      | 1        |
| 4        | UNKNOWN      |                        | 3        |

(4 строки)

```

postgres=# INSERT INTO Orders (product_code, quantity) VALUES
postgres=# ('Hello', 1);
ПРЕДУПРЕЖДЕНИЕ: Код товара "Hello" не найден в словаре
INSERT 0 1
postgres=# SELECT * FROM Orders;
Текущая кодовая страница: 1251

```

| order_id | product_code | product_name           | quantity |
|----------|--------------|------------------------|----------|
| 1        | NB-001       | Ноутбук Lenovo IdeaPad | 1        |
| 2        | PH-002       | Смартфон iPhone 14     | 2        |
| 3        | TV-001       | Телевизор LG OLED      | 1        |
| 4        | UNKNOWN      |                        | 3        |
| 5        | Hello        |                        | 1        |

(5 строк)

5. Реализуйте триггер, который использует по крайней мере 2-3 специальных переменных (NEW, OLD, TG\_OP и др).

Напишем такой триггер, который поможет отслеживать изменения в уже созданной таблице Books.

-- Создание таблицы для логирования

```
CREATE TABLE BookAuditLog (  
  id SERIAL PRIMARY KEY,  
  operation TEXT,  
  old_data TEXT,  
  new_data TEXT,  
  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Функция-триггер

```
CREATE OR REPLACE FUNCTION trigger_audit_log()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  IF TG_OP = 'INSERT' THEN  
    INSERT INTO BookAuditLog (operation, new_data)  
    VALUES ('INSERT', NEW::TEXT);  
    RETURN NEW;  
  ELSIF TG_OP = 'UPDATE' THEN  
    INSERT INTO BookAuditLog (operation, old_data, new_data)  
    VALUES ('UPDATE', OLD::TEXT, NEW::TEXT);  
    RETURN NEW;  
  ELSIF TG_OP = 'DELETE' THEN  
    INSERT INTO BookAuditLog (operation, old_data)  
    VALUES ('DELETE', OLD::TEXT);  
    RETURN OLD;  
  END IF;  
END;  
$$;
```

-- Создание триггера

```
CREATE TRIGGER trg_audit_log_books  
AFTER INSERT OR UPDATE OR DELETE ON Books  
FOR EACH ROW  
EXECUTE FUNCTION trigger_audit_log();
```

Вот так работает этот триггер:

```
postgres=# INSERT INTO Books (title, author_id) VALUES ('Дубровский', 5);
INSERT 0 1
```

```
postgres=# SELECT * FROM BookAuditLog;
```

```
Текущая кодовая страница: 1251
```

| id | operation | old_data | new_data          | changed_at                 |
|----|-----------|----------|-------------------|----------------------------|
| 1  | INSERT    |          | (23,Дубровский,5) | 2025-03-25 14:34:36.026729 |

(1 строка)

```
postgres=# DELETE FROM Books WHERE title = 'Дубровский';
DELETE 1
```

```
postgres=# SELECT * FROM BookAuditLog;
```

```
Текущая кодовая страница: 1251
```

| id | operation | old_data          | new_data          | changed_at                 |
|----|-----------|-------------------|-------------------|----------------------------|
| 1  | INSERT    |                   | (23,Дубровский,5) | 2025-03-25 14:34:36.026729 |
| 2  | DELETE    | (23,Дубровский,5) |                   | 2025-03-25 14:36:18.083881 |

(2 строки)

```
postgres=# INSERT INTO Books (title, author_id) VALUES ('Дубровский', 1);
INSERT 0 1
```

```
postgres=# UPDATE Books SET author_id = 5 WHERE title = 'Дубровский';
UPDATE 1
```

```
postgres=# SELECT * FROM BookAuditLog;
```

```
Текущая кодовая страница: 1251
```

| id | operation | old_data          | new_data          | changed_at                 |
|----|-----------|-------------------|-------------------|----------------------------|
| 1  | INSERT    |                   | (23,Дубровский,5) | 2025-03-25 14:34:36.026729 |
| 2  | DELETE    | (23,Дубровский,5) |                   | 2025-03-25 14:36:18.083881 |
| 3  | INSERT    |                   | (24,Дубровский,1) | 2025-03-25 14:37:16.955267 |
| 4  | UPDATE    | (24,Дубровский,1) | (24,Дубровский,5) | 2025-03-25 14:38:18.634841 |

(4 строки)

## Вывод

В этой работе были получены навыки работы с процедурами, функциями и триггерами на языке SQL в PostgreSQL. Основные наблюдения:

- Процедуры ничего не возвращают. Они просто проделывают необходимые команды;
- Функции выполняют команды и возвращают что-то;
- Триггер – это специальная функция в базе данных, которая автоматически выполняется при наступлении определённого события (например, вставка, обновление или удаление данных в таблице);
- Для вызова процедуры используется оператор CALL и только он;
- Для вызова функции используется оператор SELECT;
- Триггер достаточно только создать, а потом он будет применяться автоматически (в зависимости от того, какая операция выполняется над введенными данными).